

Asymptotic 1.8.0

Mathematical and engineering audit

A comparison with Wolfram Language, concrete defects, reproducible checks, and a development roadmap

Repository: VladimirReshetnikov/Asymptotic

Reviewed revision: 07a9781212beb2eeb9ff16aa625b50ac27974078

Review date: September 9, 2026

Prepared for: Vladimir Reshetnikov

Abstract

The repository is a substantial, specialized extension to Wolfram Language, not a replacement for its asymptotic engine. Its principal contribution is a reusable representation of selected real asymptotic germs: complete blocks with exact real exponents, explicit error scales, source and target branches, retained computation state, and specialized inverse constructions. Much of its value lies in preserving meaning during subsequent operations rather than merely printing an expansion.

This review traces the central power–logarithmic engine and important surrounding layers, compares the design with current official Wolfram documentation, and identifies four particularly actionable issues: a nested-observable real-branch guard bypass; an unbounded eager dense allocation during optional native export; disclosed loss of logarithmic error information in that export; and use of ambient assumptions without reliably recording their provenance. Other findings concern release validation, resource accounting, heterogeneous cutoff semantics, compatibility, and incomplete closure between supported scales.

The accompanying Python checks establish 42 mathematical and allocation-policy assertions. Six further tests exercise the patch generator on synthetic fixtures. Fifteen prospective Wolfram regression tests and a benchmark harness are supplied, but *no native Wolfram execution was available during this review*. Source deductions, independent checks, repository-reported validation, and unexecuted proposals are distinguished throughout.

Contents

1	Executive assessment and scope	4
1.1	Overall verdict	4
1.2	Priority findings	4
1.3	Revision and evidence boundary	5
1.4	What should not be called a defect	5
2	Architecture and implemented capabilities	6
2.1	The central representation	6
2.2	Inversion and reuse	6
2.3	Specialized families	6
2.4	Native delegation is part of the design	7
2.5	Three distinct kinds of checking	8
3	Mathematical analysis of the core	8
3.1	Why complete blocks are the correct unit	8
3.2	The Euler-operator coefficient formula	8
3.3	Independent coefficient checks	9
3.4	Precision algebra and its invariants	10
3.5	Inverse error transport	10
3.6	Differentiation requires additional information	11
3.7	Oscillatory and flat scales	11
3.8	Gamma and Barnes cores	11
3.9	What the exact certificate proves	12
4	Comparison with built-in Wolfram Language	12
4.1	The appropriate comparison	12
4.2	Structured data versus a durable asymptotic germ	14
4.3	Different meanings of order	14
4.4	Practical baseline calls	14
4.5	Native conditions and package assumptions	15
4.6	Recommended positioning	15
5	Concrete correctness and interoperability findings	15
5.1	A01: nested observable powers bypass the real-branch guard	15
5.2	A02: eager native export defeats sparse resource limits	16
5.3	A03: native conversion loses logarithmic error information	17
5.4	A04: ambient assumptions can escape result provenance	18
5.5	Patch scope and release caution	19
6	Engineering, API, and UX findings	19
6.1	A05: release evidence is narrower than the changed surface	19
6.2	A06: one resource option has several meanings	19
6.3	A07: cutoff and term-goal semantics are too heterogeneous	20
6.4	A08: the result-head rename is a compatibility break	20
6.5	A09: a certificate option default that cannot be used	20
6.6	A10: packet documentation integration is incomplete	21
6.7	A11: numerical application is not a checked evaluator	21
6.8	A12: shared result head does not imply operation closure	21
6.9	A13: provenance and retained state need a storage policy	21
6.10	A14: distribution and metadata consistency	22
7	Performance analysis and benchmark design	22
7.1	Improvements already present	22
7.2	Priority cost centers	23
7.3	A method-selection cost model	23
7.4	Benchmark protocol	24
7.5	Recommended counters	24

8	Validation strategy and adversarial testing	24
8.1	What was actually executed for this audit	24
8.2	Tests for invariants, not merely example outputs	25
8.3	Metamorphic and differential checks	25
8.4	Adversarial case families	25
8.5	Native runner and release gates	26
8.6	How to interpret the proposed regressions	26
9	Proposed architecture and development roadmap	27
9.1	A small trusted contract layer	27
9.2	An explicit error-contract algebra	27
9.3	Captured assumptions and proof context	27
9.4	A clean native adapter boundary	28
9.5	Capabilities and request normalization	28
9.6	Development stages and acceptance criteria	28
9.7	High-value mathematical extensions	29
9.8	A recommended product description	29
10	Accompanying artifacts and reproduction	30
10.1	Bundle contents	30
10.2	Re-running the executed checks	30
10.3	Generating the proposed patch	30
10.4	Running native regressions later	31
10.5	Rebuilding the article	31
10.6	Final assessment	31
A	Source coverage and limitations	32
	Sources and references	34

1 Executive assessment and scope

1.1 Overall verdict

Keep the mathematical core and strengthen its contracts before broadening the public surface. The package has a coherent niche: constructing and manipulating selected real inverse expansions whose exponents, logarithmic coefficient blocks, exact carriers, and branch conditions do not fit comfortably into a single ordinary power-series object. The source already contains several features that an elementary implementation would lack: complete collision handling, sparse product pruning, derivative-aware remainder policies, guarded arithmetic normalization, independent inverse residual checks, and exact rational root enclosures. These are substantive strengths, not merely interface embellishments.[\[5, 6, 7, 8, 9\]](#)

The highest-priority defects are at boundaries between those layers. A stronger check in a public wrapper is ineffective when an internal recursive route bypasses it. A sparse algorithm ceases to be resource-safe when optional metadata allocates a dense array. An expansion is not self-contained when simplifications depend on assumptions that the result does not retain. These observations motivate the main recommendation: make branch, precision, assumptions, and resource limits invariants of the underlying operations rather than conventions enforced by selected entry points.

The current implementation should be described as a collection of supported asymptotic calculi sharing a result head. It is not a general closed transseries field, a general complex-branch solver, or a uniform asymptotic solver for moving parameters. Its guide generally acknowledges such boundaries. The review recommends clearer capability discovery rather than treating every conservative refusal as a bug.[\[14, 5, 11, 10, 13\]](#)

1.2 Priority findings

ID	Priority	Finding	Required response
A01	High	Nested observable powers can accept an unknown-sign pure remainder although the direct power API rejects it.	Centralize the real-branch guard.
A02	High	Optional native export eagerly constructs a dense rational lattice without an allocation cap.	Guard before allocation; make export lazy.
A03	Medium	Native export does not preserve the logarithmic remainder degree; the guide discloses this loss.	Strict conversion or explicit loss report.
A04	High	Ambient assumptions can influence simplification without being captured in result metadata.	Capture and isolate the effective assumption context.
A05	High release priority	The recorded 1.8.0 validation is focused, not a full native suite.	Gate releases on a complete, reproducible native run.
A06– A14	Medium / design	Resource accounting, cutoff semantics, compatibility, evaluation, documentation, closure, provenance, and distribution.	Consolidate contracts and measured engineering work.

Priority denotes consequence and release importance, not an estimate of exploitability. A02 is a local availability/resource defect; no remote attack or security compromise was demonstrated. A03

is an interoperability hazard with an existing warning, not corruption of the authoritative package remainder.

1.3 Revision and evidence boundary

The review is pinned to commit `07a9781212beb2eeb9ff16aa625b50ac27974078`, whose package metadata declares version 1.8.0, MIT licensing, and Wolfram Language 15.0 or later. The branch response identifies a commit timestamp of September 9, 2026, 19:01:30 UTC. A changing `main` branch was not used as the definition of the reviewed behavior. All repository references in the bibliography point to this immutable revision. The top-level license is actually MIT No Attribution; the metadata discrepancy is discussed under A14.[1, 4, 20]

The canonical implementation is modular under `AsymptoticInverse/Kernel/`. The repository-root `AsymptoticInverse.wl` is a generated standalone distribution. The canonical main file is 83,094 bytes; the generated file is 573,940 bytes. The repository reports 38 canonical kernel source files and 37 documented public symbols. The generated file should not be treated as a second independent implementation.[2, 18, 15]

The inspection concentrated on the entire central engine as exposed through successive source ranges, arithmetic and observable dispatch, inverse expression handling, exact certificates, selected incremental and special-function code, the current guide, and validation infrastructure. Several other modules were inventoried or reached through dispatch without a complete line-by-line review. The mathematical article’s top-level source and stated scope were inspected; all of its included proofs were *not* independently audited. Appendix A records the coverage precisely.

Four evidence categories are used:

Source deduction. A conclusion follows from the inspected control flow and a stated mathematical invariant. This can establish a defect without a successful runtime session, but an exact console transcript is not claimed.

Independent execution. Python/SymPy or exact rational arithmetic was executed for mathematical identities, counterexamples, allocation planning, or patch mechanics. This is not execution of the package.

Repository report. A result was recorded by the repository’s maintainers and read at the pinned revision. It is attributed, not presented as a replication.

Prospective test or proposal. Code, a test, a benchmark, or an architectural change is supplied for subsequent native validation.

The connected Wolfram evaluation service failed even on a version query, and no local Wolfram kernel was available. Consequently, this review does not report newly measured package timings, native test passes, or observed built-in failures on particular exotic expressions. The supplied native scripts are intentionally useful despite that limitation: they turn the source deductions into concrete acceptance criteria.

1.4 What should not be called a defect

An unsupported branch is preferable to an unjustified real expansion. A finite Poincaré expansion need not converge. A formal residual against a finite forward model is not an interval enclosure of the original function. A bound that includes several separate error scales need not have one scalar exponent. An exact forward model need not have a terminating inverse. These distinctions already appear in the source and documentation and must survive future simplification of the API.[14, 5, 9, 13, 33]

2 Architecture and implemented capabilities

2.1 The central representation

The ordinary engine works in a positive local coordinate $u \rightarrow 0^+$. At a finite point this is $x - x_0$ or $x_0 - x$; at the two real infinities it is $1/x$ or $-1/x$. A finite jet is a sparse list of complete blocks

$$J(u) = \sum_{\beta < h} u^\beta P_\beta(\log u), \quad P_\beta \in \mathbb{R}[L],$$

together with a descriptor of the uncomputed part. Internally, the tuple `{rows, P, D}` means a finite expression plus an error of order $u^P(1 + |\log u|)^D$. Infinite precision denotes an exact finite expression in this representation. The actual domain and assumptions are additional information, not consequences of the tuple alone.[\[5, 6\]](#)

The public `GeneralizedSeries[association]` stores the finite expression, remainder descriptor, variable, and family-dependent metadata. Ordinary arithmetic is implemented through guarded upvalues and the held `SeriesNormalize` entry point. The representation can also contain an exact prefactor and offset,

$$\text{Offset} + \text{Prefactor}(J(u) + R(u)).$$

The distinction matters: a cutoff inside a correction bracket is not generally an absolute error cutoff for the entire expression.[\[6, 7, 14\]](#)

Formatting is deliberately separate from representation. Standard and traditional forms hide the wrapper but retain the original object through held interpretation boxes; edited display coefficients are not allowed to retain stale metadata. `Normal` explicitly discards the remainder. These choices address a real symbolic-computing problem: a visually familiar expression must not silently lose its uncertainty merely because it was copied into another input cell.[\[5, 14\]](#)

2.2 Inversion and reuse

The ordinary inverse starts from a normalized model with nonzero leading coefficient and leading power, positive correction gaps, and polynomial logarithmic coefficients. The implementation supplies direct multi-index Lagrange coefficients, a grouped Lagrange route, and Newton iteration in the jet algebra. It enumerates complete equal-weight layers, because several distinct multi-indices can contribute to one coefficient. Cancellation is performed before a block count is interpreted.[\[5, 8\]](#)

Incremental states retain the processed index region, frontier, completed blocks, and bounded polynomial-power caches. Newton states record their achieved precision and step cutoffs; an exact symbolic residual is checked after a nontrivial update. The state is returned by value, and cache exhaustion can cause recomputation rather than changing the mathematical result. This is a strong foundation for reproducible refinement, even though the size and serialization of retained states deserve further profiling.[\[8\]](#)

The public operation layer supports arithmetic, powers, logarithms, exponentials, selected observables, composition, truncation, refinement, and differentiation subject to scale and derivative contracts. `SeriesTruncate` discards information; `SeriesRefine` must obtain additional information from an actual source or retained computation. Those operations should never become interchangeable merely because they both take an order argument.[\[6, 7, 14\]](#)

2.3 Specialized families

Family	Implemented role	Important boundary
Ordinary power-log	Exact real weights, complete polynomial-log blocks, real local inversion and selected calculus.	Symbolic ordering and leading logarithmic factors need separate treatment.
Lambert and logarithmic charts	Recognized leading logarithmic/exponential cores and admitted logarithmic coordinate changes.	Not arbitrary nested-log closure or automatic choice of every inverse branch.
Exact-core perturbation	Retains a known inverse core and organizes perturbative corrections.	A formal marker degree is not automatically an asymptotic power cutoff.
Finite flat sectors	Selected exponentially small sectors with separate inner coefficient orders and sector operations.	Finite sector depth is not a general all-sector or Stokes analysis.
Fourier logarithmic coefficients	Bounded oscillatory modes, grouped frequencies, selected inverse coefficients and checks.	Boundedness does not imply invertibility of an oscillating leading coefficient.
Gamma / Barnes inverses	Ordered corrections about exact Lambert cores, with complete reciprocal-log coefficient polynomials.	Special inverse representations have a smaller operation closure than ordinary jets.
Native special-function adapters	Import structured native asymptotic trees, carriers, oscillatory phases, and error envelopes.	Supported function names do not guarantee every parameter regime and endpoint.
Zeta / Lerch constructions	Defining-sum expansions in their documented special coordinates.	Their order conventions differ from ordinary local powers.

This inventory follows the current guide and the inspected dispatch and adapter code. It is not a declaration that every combination of rows in the table is implemented.[\[14, 11, 12, 13, 10\]](#)

2.4 Native delegation is part of the design

The package is not algorithmically independent of Wolfram Language. It uses **Series** for appropriate analytic or special-function expansions, **Simplify/FullSimplify** for algebraic and branch questions, and symbolic solving for selected coordinate inversions. Its value is often the normalization, admission checks, precision transport, and reusable object built around those native results. A comparison that counts every supported special function as an independently implemented expansion algorithm would overstate the repository's contribution.[\[5, 6, 13\]](#)

The native special-function importer is notably cautious. It retains nested **SeriesData** errors instead of applying **Normal** everywhere; transports absolute error through products and exponentials; requires a real-domain justification before projecting a complex-looking native expression to a real result; and accepts a finite native result without a remainder only after establishing exact equality with the source. When a native tail does not encode a logarithmic degree, it initially weakens the power bound and seeks a justified explicit frontier.[\[13\]](#)

2.5 Three distinct kinds of checking

`InverseResidual` checks formal composition against the stored model, with explicit special-family variants. `InverseNumericalCheck` compares with a numerical reference on the selected branch. `InverseCertificate` can establish an exact rational root enclosure for its admitted elementary expression language and a supplied interval. None is a synonym for the others.[5, 9]

This hierarchy is a valuable design asset. It should become more visible in result summaries and documentation, rather than being flattened into a generic “verified” flag. A numerical comparison can corroborate a formula but cannot certify an omitted asymptotic family. Conversely, a local exact interval proof can certify a root without proving that the root belongs to the intended global inverse continuation. The certificate code explicitly limits its scope to the stored function, the selected source side, retained domain conditions, and the verification interval.[9]

3 Mathematical analysis of the core

3.1 Why complete blocks are the correct unit

Let $L = \log u$ and consider a finite set of terms $u^{\beta_j} P_j(L)$. For distinct real exponents, a smaller exponent dominates a larger one as $u \rightarrow 0^+$, regardless of any fixed polynomial logarithmic degree. At a fixed exponent, however, logarithmic degree and cancellation matter. Thus

$$u^2(L^2 + L) - u^2L^2 = u^2L$$

is one surviving block, not two terms of unrelated order. Similarly, terms arising from different multi-indices can cancel their entire polynomial coefficient. An algorithm that truncates or counts contributions before merging equal weights can return an incorrect term count and an incorrect frontier.

The package’s use of exact exponent canonicalization, complete polynomial coefficients, and layer-based merging is mathematically well motivated. For algebraic exponents, root reduction supports exact equality and ordering. For more general exact constants, simplification and sign decisions may be harder; a failure to prove an order should not be replaced by a floating-point comparison.[5, 8]

For positive gaps $\delta_1, \dots, \delta_m$, the set

$$\{k \in \mathbb{N}^m : k \cdot \delta < H\}$$

is finite for every finite H . This is the elementary reason that the finitely generated positive-weight model supports exact finite truncation even when some weights are irrational. Rational dependence creates collisions; irrationality does not itself prevent a finite computation. Uniform behavior as a parameter causes a gap to approach zero is a different problem, addressed in the roadmap rather than assumed here.

3.2 The Euler-operator coefficient formula

Write the forward model as

$$f(u) = y_0 + au^p \left(1 + \sum_{i=1}^m u^{\delta_i} P_i(\log u) \right), \quad a \neq 0, \quad p \neq 0, \quad \delta_i > 0. \quad (1)$$

On a selected real branch define $v = y - y_0$ when the target limit is finite, and use the corresponding unshifted signed target at infinity. The positive uniformizer is

$$z = (v/a)^{1/p}, \quad z \rightarrow 0^+.$$

For a fixed observable power r , expand u^r about z^r . If

$$n = |k|, \quad w = k \cdot \delta, \quad Q_k(L) = \prod_i P_i(L)^{k_i},$$

the implementation's nonzero multi-index coefficient is

$$C_k(L) = \frac{(-1)^{nr}}{p^n \prod_i k_i!} \prod_{j=1}^{n-1} \left(\frac{d}{dL} + r + w + pj \right) Q_k(L), \quad n \geq 1, \quad (2)$$

with $C_0 = 1$. Consequently,

$$u^r = z^r \sum_k z^{k \cdot \delta} C_k(\log z) \quad (3)$$

in the formal marker expansion, and as an asymptotic expansion under the requisite local regularity and remainder hypotheses. This is the formula implemented by `lagrangeCoefficient` and its cached counterpart.[5, 8, 19]

Proposition 3.1. *The coefficient formula (2) follows from perturbative Lagrange inversion with the leading monomial retained as the exact core.*

Proof. Introduce independent markers t_i in the corrections of (1), divide by a , and regard $s = v/a$ as the target. The inverse of the unperturbed map $u \mapsto u^p$ is $z = s^{1/p}$. The coefficient of total marker degree n in an observable u^r is obtained by applying $n-1$ target derivatives to the product of its core derivative and the n perturbation factors. For a fixed multi-index k , the multinomial coefficient converts $1/n!$ to $1/\prod_i k_i!$. The starting product is

$$\frac{r}{p} z^{r-p} z^{pn+w} Q_k(\log z) = \frac{r}{p} z^{r+p(n-1)+w} Q_k(L).$$

Since

$$\frac{d}{ds} (z^A Q(L)) = \frac{1}{p} z^{A-p} \left(A + \frac{d}{dL} \right) Q(L),$$

the $n-1$ derivatives give the factors with constants $r+w+p(n-1), \dots, r+w+p$. These operators commute, and their product yields (2), including the sign $(-1)^n$. This derivation establishes the formal coefficient identity; analytic convergence or finite-order error control requires separate hypotheses. $\square$

A computational advantage follows immediately: for fixed (n, w) , every contributing multi-index has the same Euler-operator product. Summing the weighted products $Q_k/\prod k_i!$ before differentiating can avoid repeated polynomial work. This explains the grouped Lagrange route; it is not an arbitrary alternate formula.

3.3 Independent coefficient checks

For $f(u) = u + u^2$, formula (2) gives the alternating Catalan coefficients:

$$u = y - y^2 + 2y^3 - 5y^4 + 14y^5 - 42y^6 + \dots$$

For $f(u) = u + u^\alpha$ with $\alpha > 1$, it gives

$$u = y - y^\alpha + \alpha y^{2\alpha-1} - \frac{\alpha(3\alpha-1)}{2} y^{3\alpha-2} + \dots$$

In particular, $\alpha = \sqrt{2}$ requires no rational exponent lattice. For $f(u) = u + u^2(1 + \log u)$, the first complete logarithmic blocks are

$$u = y - y^2(1 + \log y)$$

$$+ y^3(2 \log^2 y + 5 \log y + 3) + \dots$$

The accompanying Python program checks these identities, the $p = 2$ Puiseux case, and collision sums for $u + u^2 + u^3$ against independent ordinary Lagrange coefficients. These are mathematical cross-checks, not tests of Wolfram evaluation, package parsing, or native object formatting.

3.4 Precision algebra and its invariants

Suppose A and B are finite retained expressions with errors R and S . Then

$$(A + R)(B + S) = AB + AS + BR + RS.$$

The error order must account for the leading size of each retained expression, both input errors, and any newly discarded boundary products. The code’s precision-aware multiplication does this through valuations, leading logarithmic degrees, and an explicit boundary-degree scan. The recent optimization to a monotone boundary walk changes the cost of finding that degree, not the underlying error identity.[5]

Addition combines the weaker of the two error classes. It cannot generally improve a remainder simply because the visible leading terms cancel. For example, two independently represented quantities $u + \mathcal{O}(u^2)$ and $-u + \mathcal{O}(u^2)$ add to $\mathcal{O}(u^2)$, not exact zero. On the other hand, an expression that uses the very same exact algebraic object may have exact identities. The normalizer’s treatment of evaluation and cancellation must therefore respect both symbolic identities and uncertainty; conservative loss of a correlation is different from inventing precision.[7]

For a positive leading monomial $A \sim cu^\alpha$, $c > 0$, a differentiable fixed power satisfies

$$(A + R)^r - A^r = \mathcal{O}(u^{\alpha(r-1)}R)$$

when the relative error is small. Without a sign or branch hypothesis this statement cannot serve as a real-power rule. A pure magnitude remainder $R = \mathcal{O}(u^P)$ is especially dangerous: it does not reveal whether R is positive, negative, identically zero, or sign-changing. Finding A01 occurs precisely at that logical boundary.

3.5 Inverse error transport

Assume a real branch with

$$f'_0(u) \sim apu^{p-1}, \quad f(u) = f_0(u) + \mathcal{O}(u^\rho(1 + |\log u|)^k),$$

with enough derivative control to ensure local invertibility and stable comparison of the selected inverse branches. Let g_0 and g be the corresponding local inverses. A mean-value argument gives

$$g(y) - g_0(y) = \mathcal{O}(u^{\rho-p+1}(1 + |\log u|)^k). \quad (4)$$

For the observable u^r , the derivative contributes another factor u^{r-1} :

$$g(y)^r - g_0(y)^r = \mathcal{O}(u^{\rho-p+r}(1 + |\log u|)^k). \quad (5)$$

In the package’s positive small target coordinate this corresponds to an exponent divided by $|p|$, with the appropriate internal sign of the observable power for a source infinity. This explains why a forward remainder can cap an inverse request even when many formal coefficients are computable.[5, 14]

The guide’s “InputRemainder” $\rightarrow \{\text{rho}, \text{k}\}$ is stronger than a bare pointwise Big-O declaration: it includes a matching first-derivative order. That convention is mathematically necessary for the advertised inverse transport. It should remain explicit in any redesigned request object rather than being shortened to “input error order.”[14]

3.6 Differentiation requires additional information

A simple counterexample shows why arbitrary magnitude remainders cannot be differentiated:

$$R(u) = u^2 \sin(u^{-3}) = \mathcal{O}(u^2),$$

but

$$R'(u) = 2u \sin(u^{-3}) - 3u^{-2} \cos(u^{-3})$$

is not $\mathcal{O}(u)$. Thus `RemainderDerivativeOrder` is a mathematical contract, not merely a performance hint. A constructor with an analytically justified differentiable expansion may supply it; a user may declare it with responsibility for the hypothesis; an unrelated pure Big-O input does not imply it. The independent checks verify the derivative identity.[6, 14]

The same principle extends to other operations. Exponentiation needs a vanishing absolute error in the exponent if it is to produce a small relative multiplicative error. Keeping a large exponential carrier exact is sound; exponentiating a rough logarithmic-depth approximation without checking its absolute error can be catastrophically unsound. The inspected source-coordinate and exponential operation paths explicitly guard this issue.[6, 11]

3.7 Oscillatory and flat scales

An oscillatory coefficient such as $\cos(\log u)$ is bounded, but it has zeros arbitrarily close to $u = 0$. Consequently, an absolute envelope may remain valid while a relative error or reciprocal expansion does not. A general algorithm cannot promote “bounded coefficient” to “nonzero leading unit.” The native importer preserves oscillations inside bounded coefficients and uses separate absolute errors rather than relying on such a promotion.[13]

An exponentially small sector, for example $e^{-1/u}$, is smaller than every positive power of u . A finite number of algebraic terms cannot reveal it. Conversely, a finite-sector expansion with a sector-degree cutoff is not automatically complete within each sector’s algebraic coefficient expansion. The repository’s distinction between sector depth and inner cutoff is essential. A more unified API should expose both quantities, not disguise them as the same integer order.[14, 19]

3.8 Gamma and Barnes cores

The Gamma inverse code solves its leading logarithmic balance exactly in terms of a Lambert core. If T denotes the logarithmic target for Gamma, or the target for `LogGamma`, then

$$X(\log X - 1) = T, \quad X = \frac{T}{W(T/e)}.$$

Writing the inverse as a series in $1/X$ with whole polynomial coefficients in $1/\log X$ preserves the scale separation between algebraic corrections and logarithmic substructure. The first normalized correction in the inspected recurrence is

$$x = X + \frac{1}{2} - \frac{\log(2\pi)}{2\log X} + \dots$$

for the unshifted positive Gamma argument. Affine source transformations are handled separately.[12, 34]

For Barnes G, the leading logarithmic model in the shifted argument X gives

$$\frac{1}{2}X^2 \left(\log X - \frac{3}{2} \right) = T, \quad X = \sqrt{\frac{4T}{W(4T/e^3)}}.$$

The original Barnes argument is shifted by one; the natural reciprocal-log coefficient coordinate in the implementation is $1/(\log X - 1)$. These details are easy to lose in an oversimplified “inverse of Gamma-like growth” routine.[12, 35]

Retaining the Lambert core does not turn Stirling or Barnes asymptotic series into convergent series. The implementation correctly identifies these as finite-order Poincaré constructions. A fixed finite expansion of W in nested logarithms should not be substituted casually: its error can be much larger than the inverse-power corrections that the algorithm is attempting to retain. A future fully nested implementation must carry that extra error through the observable, not merely replace one printed expression with another.[12, 34, 35]

3.9 What the exact certificate proves

Let F be the stored explicit function, y an exact target, $I = [a, b]$ a verification interval, and $c \in (a, b)$ a center. Suppose exact interval evaluation establishes continuity on I , a constant derivative sign, and

$$|F'(x)| \geq \mu > 0 \quad (x \in I), \quad |F(c) - y| \leq \varepsilon.$$

If $[c - \varepsilon/\mu, c + \varepsilon/\mu] \subseteq I$, monotonicity and the derivative bound establish a unique root in that bracket. Alternatively, exact endpoint signs can establish existence in I first. A second mean-value enclosure using the signed interval for F' can sharpen the root interval. This is the structure of the inspected certificate routine.[9]

The implementation’s final proof arithmetic uses exact rational endpoints, directed dyadic rounding, and explicit elementary exponential/logarithm tails. Floating-point computation chooses a seed only. Unsupported predicates or expression forms are not silently accepted. A declared asymptotic input remainder is not enclosed by the certificate; the result explicitly says that it certifies the stored expression rather than every member of an unspecified remainder family.[9]

The independent bundle checks a simple example without calling the package. For $F(x) = x + x^2$, $y = 1/10$, $c = 9/100$, and $I = [2/25, 1/10]$,

$$F(c) - y = -\frac{19}{10000}, \quad F'(I) = \left[\frac{29}{25}, \frac{6}{5} \right].$$

The error radius is $19/11600$, and signed derivative sharpening yields

$$x_* \in \left[\frac{1099}{12000}, \frac{1063}{11600} \right].$$

This demonstrates the certificate argument, not independent verification of every branch of the repository’s interval evaluator.

4 Comparison with built-in Wolfram Language

4.1 The appropriate comparison

The relevant baseline is not only **Series**. It includes native structured series and their calculus, **Asymptotic**, implicit asymptotic equation solving, asymptotic comparison predicates, and discrete asymptotics. Their documented scope is broader than ordinary Taylor expansion. The comparison below concerns documented capabilities and inspected package design, not an experimentally established ranking of success rates.[21, 22, 23, 24, 25, 26, 27, 28]

Task	Native Wolfram Language	Repository contribution / limitation
Taylor, Laurent, Puiseux	Series and SeriesData ; native arithmetic and calculus.	Overlapping functionality; extra branch, source, and explicit envelope metadata.
Logarithmic behavior	Native series may have logarithmic or other slowly varying coefficients.	Complete polynomial-log blocks with an explicit logarithmic remainder degree.
Noncommensurate real powers	One SeriesData uses a common rational exponent lattice; broader native expressions can contain external factors and exotic scales.	Sparse exact real exponent weights with complete collision grouping in an admitted real class.
Reversion	InverseSeries for suitable structured series; AsymptoticSolve for implicit equations.	Explicit local source side, multi-index coefficients, selected exact cores, retained inverse state.
Composition	ComposeSeries and native series substitution with compatible centers.	Selected generalized-scale composition, explicit error transport, guarded failures outside closure.
General asymptotics	Asymptotic infers scales and handles a broad range of objects.	More specialized constructors with durable result objects; not a replacement for the full native scope.
Real and complex branches	AsymptoticSolve supports real-valued solutions and complex approaches.	Deliberately selected real branches; branch metadata is a principal feature.
Special functions	Broad built-in expansion support.	Many forward paths delegate to native Series and import its structured errors; Gamma/Barnes inverse organization adds specialized algorithms.
Oscillation / exponentials	Native results can contain exact rapidly varying factors outside a series object.	Selected bounded Fourier coefficients, exact carriers, finite flat sectors, absolute error envelopes.
Discrete problems	DiscreteAsymptotic covers integer-variable asymptotics.	No inspected general-purpose replacement for this discrete problem family.
Numerical rigor	Ordinary symbolic approximation and numerical root finding are not themselves the package's exact residual certificate.	An explicit, restricted rational-interval certificate API with a narrowly stated scope.

The native entries are grounded in the official references; the package entries are grounded in its guide and implementation. Claims of a universal inability of Wolfram Language to handle logarithms, exponentials, irrational powers, or asymptotic inverses would be incorrect.[[21](#), [22](#), [23](#), [24](#), [25](#), [26](#), [27](#), [14](#), [5](#), [13](#), [12](#), [9](#)]

4.2 Structured data versus a durable asymptotic germ

A native `SeriesData` has a rational lattice encoded by integer indices and a denominator. It participates naturally in arithmetic, function application, differentiation, integration, and composition. Its surrounding expression can contain logarithmic coefficients or external non-power factors. Those facilities already cover a large fraction of routine local series work.[22, 24]

The repository's differentiator is the *combination* of an admitted generalized support with a stored positive coordinate, branch conditions, complete error descriptor, and reconstruction or refinement recipe. The result can be interrogated after the original computation. However, that claim is only as strong as the invariants of every operation and metadata transfer. Findings A01 and A04 show why it is insufficient to attach a descriptive association after a successful symbolic computation.

4.3 Different meanings of order

For ordinary integer-power local expansions, `Series[f,{x,0,n}]` requests a polynomial order through n , while the package's ordinary cutoff h is exclusive: retain weights strictly smaller than h . Thus $n = h - 1$ is a useful matching rule only in the ordinary integer-power setting. It is not a universal conversion for relative prefactors, irrational weights, logarithmic hierarchies, or sector depth.[21, 14]

Native `Asymptotic` and `AsymptoticSolve` interpret approximation order and `SeriesTermGoal` in their inferred asymptotic scales, not as the package's universal power-weight cutoff. The package counts complete nonzero blocks; a structured multi-carrier forward result can apply its goal separately to each carrier. A benchmark that assigns the same integer to every function without checking the actual retained information is not a fair comparison.[25, 26, 14]

4.4 Practical baseline calls

The following are comparison inputs, not newly executed results:

```
(* Native local inverse, with its truncation made explicit. *)
InverseSeries[x + x^2 + O[x]^5, y]

(* Package: exclusive target-power cutoff. *)
AsymptoticInverse[AsymptoticInverse[x + x^2,
  {x, 0}, {y, 5}]

(* Native implicit route: selected point and real-valued solutions. *)
AsymptoticSolve[x + x^Sqrt[2] == y, x -> 0, y -> 0,
  Reals, Assumptions -> y > 0, SeriesTermGoal -> 5,
  GenerateConditions -> True]

(* Package: five complete generalized blocks on its source side. *)
AsymptoticInverse[AsymptoticInverse[x + x^Sqrt[2],
  {x, 0}, y, SeriesTermGoal -> 5]
```

The first pair has an analytically known common finite inverse polynomial. The second pair asks related, not automatically identical, questions: native scale inference, branch reporting, output structure, and term conventions must be inspected before claiming equivalence. The official implicit solver also supports systems and more general approaches than the package's ordinary scalar real interface.[23, 26]

4.5 Native conditions and package assumptions

Native `Asymptotic` and `AsymptoticSolve` offer condition-generation options. A comparison should enable the needed conditions rather than suppress them and then criticize the native result for lacking a branch guarantee. Conversely, the package should not be credited with stronger provenance merely because it has an `"Assumptions"` property; that property must contain every assumption actually used.[25, 26, 14]

The ambient-assumption issue in Section 5.4 is particularly relevant to integration with normal Wolfram workflows. `Assuming` is a standard way to supply assumptions, and simplification functions may use the global assumption context. A durable result should either capture that context explicitly or deliberately isolate itself from it. Mixing the two policies is the defect.[30, 31]

4.6 Recommended positioning

Use native structured series for routine commensurate local expansions, native implicit asymptotics for broad equation-solving tasks, and native discrete asymptotics for their integer-variable domain. Use this package when complete generalized blocks, selected real inverse cores, transported error metadata, repeated refinement, or the explicit certificate workflow are central to the task. This is an engineering recommendation based on scope, not a measured rule that one engine is always faster or more successful.

The best long-term architecture is cooperative: native engines provide admitted local expansions and symbolic subroutines; the package supplies a coherent asymptotic-germ contract, sparse generalized support, provenance, and checked transformations. A more explicit adapter boundary will make failures diagnosable and help avoid duplicate implementations of features already provided reliably by the host language.

5 Concrete correctness and interoperability findings

5.1 A01: nested observable powers bypass the real-branch guard

Evidence: High-priority source deduction, supported by an elementary counterexample; native reproduction not executed.

Location. `AsymptoticInverse/Kernel/SeriesOperations.wl`, the direct `seriesPower` guard and recursive `seriesJetApply`; `AsymptoticInverse/Kernel/AsymptoticInverse.wl`, `fwdPower`. The relevant inspected windows are lines 165–190 and 181–320 of the former, and 354–405 of the latter.[6, 5]

The direct operation rejects a noninteger power when an operand consists only of an uncertain remainder. That is correct: no real branch can be inferred from a magnitude class alone. However, a nested power inside a general observable is dispatched straight to `fwdPower`. Its empty-jet branch permits any positive noninteger exponent by multiplying the error exponent and rounding up the logarithmic degree, without establishing a real sign.

A minimal public-path test is:

```
Clear[x, z];
s = AsymptoticExpansion[-x^2, {x, 0, 2}];
SeriesPower[s, 1/2]
SeriesObservable[s, 1 + Sqrt[z], z]
```

The first construction discards the weight-two block and retains a pure $\mathcal{O}(x^2)$ remainder. The direct power route should return a failure and has an explicit guard to do so. The recursive observable

route instead reaches the unguarded empty-jet power rule; the predicted finite result is 1 with an error of order x . This is a control-flow prediction, not a captured Wolfram output transcript.

Why it is wrong. The original real function is $-x^2$ on $x > 0$, and no real square root exists. Its complex principal square root ix does satisfy a complex magnitude bound of order x , so this finding is *not* a claim that the complex modulus estimate is numerically false. It is a violation of the API's real-branch contract. More generally, the same remainder class includes both x^2 and $-x^2$; a source-independent real observable cannot decide the square-root branch from that class.

Fix. Move the invariant into `fwdPower`: for an empty jet with finite uncertainty and a noninteger exponent, return `Failure["UnknownLeadingTerm", ...]` unless an explicit sign/domain certificate is supplied. Preserve the exact-zero case and known positive leading monomials. Public forward construction may respond by increasing working order until a genuine leading term is available. Arithmetic on a source-less uncertain object may not invent that information.

The supplied patch generator implements the conservative rejection, not a new sign-proof subsystem. Tests cover the direct and nested routes, a known positive base, and exact zero. The fix should also be exercised through inverse target composition and nested analytic functions because those reuse the same low-level evaluator.

5.2 A02: eager native export defeats sparse resource limits

Evidence: High-priority source deduction with an independently checked allocation count; no enormous allocation was executed.

Location. The main kernel's `makeSeriesData` and `makeInverseSeriesData`, called while constructing result associations. Their inspected windows are 621–760 and 901–1040.[5]

The internal representation is sparse, but the optional `"SeriesData"` field is computed eagerly. The conversion obtains a common denominator, converts retained and remainder weights to integer indices, then allocates

```
coeffs = Table[0, {nmax - nmin}];
```

without first limiting the number of slots. A small number of blocks therefore does not imply a small conversion.

Consider, for an exact positive integer N ,

```
With[{n = 1009},
  AsymptoticExpansion[x^(1/n) + x, {x, 0, 1},
    "MaxTerms" -> 10]]
```

where the displayed call uses the safe concrete value $N = 1009$. Only the weight $1/N$ is retained, and the first omitted weight is 1. The common lattice denominator is N , so $n_{\min} = 1$, $n_{\max} = N$, and the temporary coefficient array has $N - 1$ slots. At $N = 10^9$ this is 999,999,999 slots, despite the tiny sparse input. The independent checker calculates that cardinality without allocating the array.

A native series constructor may later remove trailing zero coefficients. That does not cure the defect: the temporary dense `Table` has already been allocated. Accordingly, this review does not claim that the final exported object's coefficient list necessarily remains that large.

Impact. A mathematically trivial sparse request can exhaust memory or spend excessive time during metadata construction. The user-supplied `"MaxTerms"` does not constrain that allocation. Since the export is optional interoperability data, its failure should not invalidate the already available generalized expansion.

Fix. Compute a conversion plan first. It should contain the lattice denominator, minimum and maximum indices, required slots, and reasons why conversion is lossy or unavailable. Refuse or defer dense export when its budget is exceeded. The preferred architecture computes "SeriesData" lazily on request and returns `Missing["SeriesDataSizeLimit", ...]` rather than attempting an uncontrolled allocation.

The prototype inserts a fixed 20,000-slot guard in both conversion functions. This is an interim safety cap, *not* complete plumbing of the caller's "MaxTerms" through all operations. A production change should expose a dedicated export-size budget or incorporate the conversion into a coherent shared resource policy. The native regression uses $N = 20011$, so the unpatched allocation is modest; the billion-slot example is never executed.

5.3 A03: native conversion loses logarithmic error information

Evidence: Disclosed interoperability limitation, independently demonstrated mathematically; a stricter policy is proposed.

Location. The same two conversion functions use the remainder power but do not encode its logarithmic degree. The current guide already warns that native `SeriesData` may hide the logarithmic degree and instructs users to keep the package remainder.[\[5, 14\]](#)

For

```
s = AsymptoticExpansion[x + x^2 Log[x], {x, 0, 2}];
{s["Remainder"], s["SeriesData"]}
```

the authoritative package object records an error of order

$$x^2(1 + |\log x|).$$

The native export is structurally of the form $x + O(x)^2$. Interpreting this as a strict mathematical $\mathcal{O}(x^2)$ error loses information, because

$$\frac{x^2 \log x}{x^2} = \log x \longrightarrow -\infty.$$

This does not establish a defect in the primary `GeneralizedSeries` remainder; that remainder is the correct one. Nor does it imply that native Wolfram series cannot contain logarithms. Native representation conventions and the surrounding coefficient expressions are richer than a plain analytic power series. The issue is the loss of the package's stronger, explicit error contract at the export boundary.[\[22, 29\]](#)

There are three defensible policies. A strict conversion can refuse nonrepresentable error information. A permissive conversion can return the native object together with a loss report and the authoritative sidecar remainder. Or a mathematically conservative conversion can weaken the exponent by a justified positive amount, using the fact that every fixed logarithmic polynomial is dominated by any inverse positive power. The last option must also account for retained terms and the new cutoff; simply editing an order index is not a general solution.

The prototype chooses strict conversion: it rejects a nonzero remainder logarithmic degree and variable-dependent retained coefficients in this optional export. The latter rejection is intentionally conservative and may remove exports that native Wolfram Language can represent in its generalized conventions. This is a policy change, not a claim that such native expressions are invalid.

Even a strict native power-series export does not encode all branch and assumption provenance. A complete semantic round trip requires a sidecar or a dedicated generalized interchange schema. The requested improvement is therefore explicit loss accounting, not a promise that one `SeriesData` object can preserve every field.

5.4 A04: ambient assumptions can escape result provenance

Evidence: High-priority source deduction grounded in documented Wolfram evaluation semantics; native reproduction not executed.

Location. The main kernel's simplification and canonicalization helpers, its default `Assumptions -> True`, and public wrappers in `InverseFunctionExpressions.wl`. The inspected source uses `Simplify/FullSimplify` with local assumption arguments and sometimes without one; it does not consistently isolate `$Assumptions` or capture the ambient context into the returned object.[\[5, 10\]](#)

Wolfram's simplification semantics matter here: the default assumption option draws on `$Assumptions`, and that option is appended to an explicitly supplied second assumption argument. `Assuming` temporarily extends the ambient assumption context. Thus passing `True` as the local assumption argument does not necessarily mean that the calculation was assumption-free.[\[31, 30, 32\]](#)

A diagnostic input is:

```
Clear[a, x];
s = Assuming[a == 0,
  AsymptoticExpansion[a x + x^2, {x, 0, 3}]];
{Normal[s], s["Assumptions"], s["Function"]}
```

Inside the construction, zero tests can discard ax under the ambient condition $a = 0$. Yet the stored assumption value is obtained from the explicit option, whose default is `True`. The predicted result therefore has finite expression x^2 while lacking the condition that made this exact simplification legitimate. The original symbolic function can remain $ax + x^2$. Outside `Assuming`, at $a = 1$ and $x = 1/10$, the discarded term is $1/10$.

The problem is not that simplification under $a = 0$ was mathematically incorrect. It is that the dependent conclusion is packaged as a reusable object without the hypothesis. The same issue can affect sign proofs: an inverse admitted under ambient $a > 0$ may outlive that condition if the object records only the default explicit assumptions. Subsequent arithmetic, refinement, or serialization can then operate on incomplete provenance.

Fix. Choose one explicit policy and apply it consistently at every public entry point:

1. Capture the effective ambient-plus-explicit assumptions at entry, record them, split approach-dependent conditions where appropriate, and run internal computation with ambient `$Assumptions` neutralized; or
2. Deliberately ignore ambient assumptions, isolate internal computation, and require the explicit option as the only admitted source of assumptions.

The first policy is more natural for ordinary Wolfram workflows. In either case, every operation on a stored object must use its retained context rather than accidentally acquiring unrelated ambient hypotheses.

A safe construction pattern for an explicitly supplied condition is:

```
Block[{$Assumptions = True},
  AsymptoticExpansion[a x + x^2, {x, 0, 3},
    Assumptions -> a == 0]]
```

This is a workaround, not a package-wide repair. Changing only the default option to `$Assumptions` is insufficient if later nested operations can still read an unrelated global context. Likewise, simplifying a captured assumption formula under itself can collapse it to `True` and recreate the provenance loss. Context capture should be structural, and proof evaluation should be isolated.

The regression bundle checks an invariant rather than an exact output string: outside the ambient context, the stored assumptions must justify equality of the finite exact result and its original

expression. A second test requires an ambient sign proof either to be retained or to be conservatively rejected. These acceptance criteria leave room for either coherent policy.

5.5 Patch scope and release caution

The provided Python patch generator is hash-gated to the canonical main-file Git blob at the reviewed revision. It prints a unified diff by default and refuses to overwrite a file. It addresses the low-level power guard, a conservative export policy, and an interim dense-export cap. It does not repair the cross-cutting assumption issue, add lazy properties, or unify resource budgets.

Only its transformation mechanics were tested on synthetic source fixtures. The real canonical file was inspected through the connector but was not assembled into a local checkout, and the proposed kernel was not loaded in a Wolfram runtime. The hash and unique-anchor checks deliberately make the script fail rather than guess when applied to a changed revision. After review and application, the canonical modular package must pass native tests and the standalone distribution must be regenerated from it.

6 Engineering, API, and UX findings

6.1 A05: release evidence is narrower than the changed surface

Evidence: Repository-reported validation scope and inspected workflow; no independent native rerun. The version 1.8.0 record reports 305 passing focused tests in 18 selected suites, with no failures in that run. It explicitly says the full suite was not run. It also records separate builder, documentation, and isolated loading checks. Those are useful results, but they should not be combined into an unsupported assertion that every package path passed on that revision.^[15]

The inspected GitHub Actions workflow checks standalone freshness and Python builder tests; it does not run Wolfram semantic tests. This is a statement about that checked-in workflow, not a claim that no external or manually operated validation exists. The risk is especially relevant when a common result head, public arithmetic, and several adapters change together. A narrow regression run can miss interactions outside its selected files.^[17]

Recommendation. Keep focused tests for development feedback, but require a complete native run before a release is declared validated. Identify the source hashes, kernel version, platform, selected suites, timeouts, empty suites, and aborted runs in a machine-readable manifest. Store failures and unsupported cases rather than only aggregate pass counts. The repository's shared focused runner already checks several of these conditions; extending that discipline to a release gate is a better starting point than replacing it.^[16]

6.2 A06: one resource option has several meanings

Evidence: Source-level design limitation; not a measured end-to-end timing regression. "MaxTerms" can govern retained pair products, index regions, cache capacity, expression leaf counts, or input operands depending on the path. Some automatic arithmetic uses a hard-coded default rather than a caller's full context. Independent hard limits also occur for Taylor work, repeated order enlargement, source-chart depth, and timed native simplification or series calls. A request can therefore fail below the advertised term count, or perform expensive work not bounded by that count.^[5, 7, 8, 11, 13]

Several local limits are prudent safeguards. The problem is their discoverability and composition, not the mere fact that they exist. A per-probe time limit does not provide an end-to-end time limit when many probes are attempted. A cap on the number of polynomial coefficients does not bound

the size of each coefficient. A budget consumed by optional caching should not reduce the set of mathematically admissible output terms; the incremental cache already takes care to avoid that particular mistake.[8]

Recommendation. Introduce a shared request budget with separate counters for support size, polynomial complexity, candidate products, dense export slots, memory estimate, simplification calls, and elapsed time. Every failure should report the exhausted dimension, used amount, requested amount where known, and last certified precision. Preserve a usable partial result when the API contract allows it, but never disguise a partial result as meeting the requested cutoff.

6.3 A07: cutoff and term-goal semantics are too heterogeneous

Evidence: Documented design complexity. The guide carefully distinguishes ordinary powers, correction brackets, Lambert inverse-logarithmic weights, Gamma/Barnes core powers, source-coordinate reconstructions, total perturbation depth, flat-sector degree, Fourier weights, and defining-sum coordinates. That accuracy is commendable, but the same positional order argument now means many things. Structured forward special functions can return more total blocks than a nominal term goal because each carrier is counted separately.[14]

A user should not need to inspect a family-specific source file to determine whether a request means absolute accuracy, relative accuracy, retained blocks, marker degree, or sector depth. Introduce a normalized request description, such as an association containing "Coordinate", "OrderKind", "Boundary", and "Inclusive", while retaining existing syntax as a convenience wrapper. Every result should expose both the original request and its resolved interpretation. A capability query should say whether a requested operation changes the scale or can honor that order convention.

6.4 A08: the result-head rename is a compatibility break

Evidence: Documented breaking interface change. Version 1.8.0 replaces `PowerLogSeries` with `GeneralizedSeries`; the guide tells users to update explicit patterns and saved input. The broader name accurately describes the supported families, but an object-head change affects notebooks, pattern-matching code, serialized expressions, and external packages. It is not only a cosmetic rename.[14, 1]

For a stable package, choose an explicit compatibility policy. A deprecated reader or constructor can migrate old objects if their schema is recognizable; alternatively, a major-version transition can clearly advertise the break. Automatic migration should not simply rename the head of an arbitrary association, because old metadata may have different precision semantics. Store a schema version, validate required invariants, and maintain archived round-trip fixtures. Do not rely solely on the package version embedded in the loading file.

6.5 A09: a certificate option default that cannot be used

Evidence: Source-confirmed API inconsistency, not a false certificate. `InverseCertificate` declares "Interval" -> `Automatic`, but its entry validation immediately requires a literal ordered pair of exact rational endpoints. Calling it without the interval fails. The documented usage supplies an interval, so the current code is conservative; the inconsistency is that the option table suggests automatic interval construction even though that behavior is not implemented.[9, 14]

Either use a clearly required/missing option value with a targeted diagnostic, or implement automatic bracket generation as a separate, verified step. Automatic bracketing must check the selected branch and all retained source-domain conditions, not merely search near the asymptotic seed with floating-point samples. Report failure to find a bracket separately from failure of the exact enclosure arithmetic.

6.6 A10: `paclet` documentation integration is incomplete

Evidence: Inspected metadata and current documentation layout. The repository has extensive Markdown/HTML user documentation and a mathematical article, but the inspected `PacletInfo.wl` registers only a kernel extension. It does not register a documentation extension. This does not mean that documentation is absent; it means the `paclet` metadata does not provide a full native documentation-center integration.[4, 14, 18]

A useful improvement is a reference page for each public symbol with evaluated examples, option semantics, failure examples, and related functions. Automatically test the examples in a fresh kernel and link each page to its mathematical contract. Give the order/coordinate explanation a common location that every constructor references. The extensive existing guide can supply the content; the task is integration and executable consistency, not writing another independent manual.

6.7 A11: numerical application is not a checked evaluator

Evidence: Documented behavior with a potentially misleading convenience syntax. The rule `s[value]` substitutes the value into the finite expression. It is not a certificate request and does not by itself establish that the numerical target lies in a valid asymptotic regime. The guide explains that it evaluates the finite expression, and `Normal` has similarly explicit information-dropping semantics.[5, 14]

Nevertheless, function-like application encourages users to interpret the result as a reliable approximation to the original inverse. Introduce a separate checked evaluator with domain validation, requested precision, a distinction between an error *scale* and an explicit error *bound*, and an optional certificate route. Keep the lightweight substitution syntax, but give it an unmistakable role in the documentation. An unspecified Big-O constant cannot be converted into a numerical error bar merely by evaluating its scale.

6.8 A12: shared result head does not imply operation closure

Evidence: Documented limitation and architectural opportunity. The package includes ordinary, logarithmic, exact-core, flat, Fourier, special inverse, and composite results. Some operations apply only to selected representations; others fall back to a composite absolute envelope. For example, a composite result does not have one exponent cutoff, and source reconstruction can fail when an intermediate exponential would amplify insufficient precision. These are often mathematically justified refusals.[7, 11, 14]

The UX problem is not that all combinations must succeed. It is that the user discovers the admissible algebra by trying operations and interpreting family-specific failures. Add `SeriesCapabilities[s]` or an equivalent property that lists supported operations, preconditions, possible representation changes, and whether refinement has a valid source. The operation layer should consult capabilities and shared invariants instead of growing a long list of kind-specific exceptions.

6.9 A13: provenance and retained state need a storage policy

Evidence: Source-level performance and maintainability hypothesis; profiling required. Objects may retain original functions, models, branch records, operation recipes, operand objects, computation states, and caches. That provenance enables refinement and auditability, but repeated composition can duplicate large symbolic structures. A compact printed expression can conceal a large underlying association. The inspected code does not justify a specific memory-growth rate, so no quantitative blow-up claim is made here.[7, 8, 10, 12]

Measure `ByteCount`, serialized size, repeated-subexpression counts, and refinement reuse. Consider a compact immutable proof/recipe graph with content-addressed nodes, optional cache dropping, and an explicit `CompactSeries` operation that preserves the mathematical contract while removing reproducible performance state. Distinguish essential evidence from optional acceleration data. Exporting a notebook should not inadvertently embed a large history of equivalent intermediate objects.

6.10 A14: distribution and metadata consistency

Evidence: Documented distribution behavior and a literal license-metadata mismatch. The quick-start command loads the current `main` standalone file, while the guide also gives a commit-pinned and offline route. The development notes document a real reason to download fully before loading: direct compressed HTTP reads were problematic in the tested kernel, and an earlier nested convenience loader was withdrawn after abnormal exits. Reintroducing an elaborate remote loader without addressing that history would be a regression in engineering discipline.[14, 18]

Prefer a pinned release artifact and checksum for reproducible work. Keep the generated standalone file deterministic and derived only from canonical modules. The existing builder validates dependency structure and checks byte-for-byte freshness, which is a good foundation. A package signature or trusted release channel would address provenance more directly than another layer of automatic source fetching. No malicious distribution incident was observed.[18, 17]

There is also a concrete metadata discrepancy: `PacletInfo.wl` says "License" -> "MIT", whereas the root `LICENSE` is headed "MIT No Attribution License." The SPDX identifier for the latter is `MIT-0`, distinct from ordinary MIT. Align the package metadata with the intended authoritative license text. This is a release-metadata correction, not a conclusion about a licensing dispute.[4, 20, 36]

7 Performance analysis and benchmark design

7.1 Improvements already present

A useful audit must not recommend work that the current revision already implements. The inspected source already has sparse truncation-aware multiplication, efficient enumeration of weighted index regions instead of a surrounding Cartesian cube, a stars-and-bars preflight for depth regions, grouped equal-weight coefficients, polynomial-power caching with bounded occupancy, and Newton precision doubling with a residual invariant.[5, 8]

The 1.8.0 validation record reports focused improvements to coordinate reconstruction, Fourier merging/products, and checks. Its timings are repository measurements, not measurements performed for this article. In particular, a very large speedup in a low-level product does not imply the same gain in a complete inverse calculation.[15]

Repository benchmark case	Before (s)	After (s)
100 translated inverse reconstructions	0.12115	0.00418
Fourier merge case	0.19112	0.09396
200 × 200 Fourier product, cutoff 4	1.28634	0.00776
Public Fourier inverse, weight 4	0.02514	0.02325

Table 4: Selected timings as recorded by the repository. The validation record describes a warm-up and three measured samples in a fresh kernel. These numbers were not replicated in this audit and are not a comparison against native Wolfram functions.[15]

7.2 Priority cost centers

Dense export. A02 is the clearest allocation problem because the cost follows directly from an exact index count. Fix it before optimizing coefficient algebra: otherwise improvements to the sparse core can be overwhelmed by metadata conversion.

Exact comparison and simplification. Exact exponent comparison, coefficient zero tests, and repeated full simplification can dominate small algebraic operations. Cache normalized exponents and proof results within an explicit immutable assumption context. A cache keyed only by an expression, without its domain and assumption context, can become mathematically unsafe. Avoid global caches that silently acquire ambient assumptions.[5, 10]

Frontier selection. The incremental inverse routine scans boundary values to select the current layer and sorts boundary weights to find the next one. A priority queue could reduce repeated full scans when the frontier is large. However, it must use exact ordering and batch all equal-weight collisions before producing a block. For small frontiers, the current list/association approach may be faster; this is a profiling candidate, not a guaranteed improvement.[8]

Gamma/Barnes coefficient rebuilding. The inspected Gamma inverse routine computes coefficients in increasing order, repeatedly expands a phase series, and may rebuild rows while searching for a nonzero frontier. Retaining a recurrence state could avoid recomputing lower-order coefficients. The stored coefficient ring is structured enough to support specialized polynomial operations in the reciprocal-log variable. A production change should first compare coefficients and frontier bounds exactly across several observable powers and affine transformations.[12]

Native adapter probes. The forward dispatcher tries several families and performs bounded native expansions and simplifications. Repeated failed probes can cost more than the successful expansion. Introduce cheap structural admission predicates, a per-request dispatch trace, and a shared time budget. Cache a negative admission result only when its dependence on parameters, direction, and assumptions is explicit.[10, 13, 11]

Object history. Retained recipes and operand trees can make copying or serialization expensive even when arithmetic is fast. Profile full public objects, not only `Normal[s]`. Caches should be optional and removable without changing the finite expression, remainder, or proof obligations.

7.3 A method-selection cost model

For m positive correction gaps and a fixed cutoff H , direct Lagrange work is related to the number of multi-indices in $k \cdot \delta < H$. Its leading large- H geometric scale is

$$\frac{H^m}{m! \prod_i \delta_i},$$

for fixed positive gaps, before polynomial coefficient costs and boundary overhead. This is a planning estimate, not an exact count or a uniform formula as gaps vary. The actual weighted region should still be enumerated or bounded exactly for resource admission.

Grouped Lagrange benefits when many indices share the same pair (n, w) and when differentiating their logarithmic polynomial products is expensive. Newton benefits from increasing precision and reusable truncated products, but its constant factors and coefficient complexity can outweigh those gains at low orders. A sensible `Method -> Automatic` should use measured structural features rather than a fixed preference: gap count, rational relations, expected collisions, logarithmic degree, requested precision, and whether a valid previous state is available.

The current Gamma/Barnes family already records both the requested ordinary method and its actual ordered special-family method. Preserve that transparency: a method option should not silently pretend to select an algorithm that is not used. Report the selected engine, the reason, and any fallback.[12]

7.4 Benchmark protocol

Every timing comparison should first establish comparable mathematical output. Normalize the source/target branch, coordinate, retained block set, exact carriers, and requested remainder strength. A fast answer with a weaker error, a different inverse branch, or fewer complete blocks is not an equivalent result.

Use a fresh kernel for each case, separate loading cost from first-call cost and warm performance, record the complete kernel/platform version, and retain all samples. The supplied harness performs one warm-up followed by three timed calls, with an explicit per-call timeout, and records **ByteCount**, **LeafCount**, and the full input/output forms. It does not automatically certify equivalence between related native and package calls.

The benchmark matrix should contain ordinary Taylor and Puiseux controls, irrational gaps, multiple colliding gaps, high-degree logarithmic coefficients, cancellation at the frontier, composition and refinement, Gamma/Barnes inverses, flat sectors, oscillatory coefficients, failed branch proofs, and very sparse rational lattices. Include both low-level kernels and the public entry point. Failed or unsupported cases are part of the result, not samples to discard.

For oscillatory numerical checks, evaluate absolute residuals scaled by a nonvanishing envelope rather than divide by a leading oscillatory term near its zeros. For logarithmic/exponential reconstructions, compare phase residuals when appropriate to avoid overflow and cancellation. For asymptotic remainder validation, a finite set of numerical points is evidence only; it is not a proof of a limiting Big-O assertion.

7.5 Recommended counters

A diagnostic result should expose the number of generated and retained support entries, merged collisions, coefficient simplifications, polynomial-power cache hits and evictions, boundary scans, native oracle calls, precision-doubling steps, reconstruction attempts, and bytes retained in the final object. Some incremental statistics already exist. Standardize them rather than introducing another disconnected performance report.^[8]

These counters also help users understand a resource failure. “MaxTerms exceeded” is much less actionable than “19,842 retained indices plus 321 frontier indices exceed a 20,000 index budget; the last certified cutoff is h .” The exact wording can vary, but the mathematical state at failure should be visible and unambiguous.

8 Validation strategy and adversarial testing

8.1 What was actually executed for this audit

Check group	Result	Scope
Independent Python/SymPy checks	42 passed	Coefficient identities, counterexamples, allocation planning, and one exact rational certificate argument.
Patch-generator unit tests	6 passed	Synthetic fixtures, anchor validation, budget handling, and Git-blob hashing.
Supplied Wolfram regressions	15 supplied; not run	Corrected-behavior tests and ordinary controls in a native kernel.
Supplied benchmark harness	Not run	Native/package comparison inputs and measurement capture.

Check group	Result	Scope
Repository 1.8.0 focused suite	305 passes reported	Maintainer-recorded 18-suite run; full suite explicitly not run.

The first two results are in the bundle’s evidence files. The last row is attributed to the repository. The Python checks are deliberately independent of Wolfram evaluation and do not pretend to emulate upvalues, holding attributes, assumptions, or `SeriesData`.^[15]

8.2 Tests for invariants, not merely example outputs

An exact expected polynomial is a useful regression test, but it does not cover the whole contract. Each result should also be tested for the validity of its coordinate, domain, remainder, exactness status, and refinement recipe. In particular, a visually correct finite expression can still have an invalid branch or a falsely strong error bound.

For every elementary operation, construct the same observable through multiple paths: the dedicated API, ordinary arithmetic, `SeriesNormalize`, nested `SeriesObservable`, and composition. The mathematical result and branch admission policy should agree, modulo explicitly documented representation changes. A01 shows why these path-equivalence tests are essential.

Assumption tests should run with explicit assumptions, ambient `Assuming`, both together, conflicting conditions, and a fresh outer context after construction. Test that a saved object can be reloaded and used without relying on an assumption that was active only in the original session. Deliberately changing `$Assumptions` after construction must not retroactively change the object’s claimed branch or exactness.

8.3 Metamorphic and differential checks

Several useful properties do not require an independent generalized-series solver. Construct a finite analytic or algebraic inverse g first, derive a forward function when feasible, and check that reversion recovers the expected germ to the requested precision. Compare Lagrange, grouped Lagrange, and Newton on their common admitted class after canonicalizing complete blocks. Compare ordinary power-series cases with native `InverseSeries` and composition with `ComposeSeries`.^[23, 24]

Test that truncating twice is consistent with truncating to the weaker final order. Test that a failed refinement leaves the original result unchanged. Check that refining a source-backed result agrees with a fresh construction at the stronger order, while an object carrying only an unknown remainder cannot gain precision. Exact termination should require exact evidence, not a long run of zero computed coefficients.

Differential tests against native `AsymptoticSolve` should compare branch conditions and asymptotic residuals, not raw expression strings. An unevaluated native call is an unsupported or unresolved case in that tested version, not proof that the host language has no relevant capability. Likewise, a package failure may be the correct response to an unproved real branch.

8.4 Adversarial case families

Cancellation and exact zero. Include zero finite parts, canceled leading blocks, exact zero, powers zero and negative, and unknown denominators. Distinguish algebraic identity from cancellation of independently uncertain operands.

Exponents and frontiers. Include rational exponents with large denominators, irrational algebraic exponents with exact relations, nearly equal numerical values that are exactly different,

and symbolic gaps under different parameter inequalities. Ensure that equality grouping precedes truncation and term counting.

Logs and nested operations. Include nonzero logarithmic remainder degree, high-degree complete log blocks, logarithms of sign-indefinite inputs, and exponentials whose argument remainder is not small in absolute value.

Branches and endpoints. Test both finite source sides, both infinities, translated coordinates, negative source powers where admitted, conditional expressions, and a target with the same limiting value reached by different source branches.

Oscillation and special functions. Test zeros of oscillatory leading coefficients, real-domain boundaries, distinct exponential carriers, cancellation between carriers, and unsupported parameter regimes that must fail rather than silently project a complex result.

Certificates. Test intervals crossing singularities or violating a source condition, strict endpoint contact, derivatives not separated from zero, fixed-center accuracy floors, relative accuracy near zero, and a declared input remainder whose family is not certified. The certificate code already treats many of these conservatively; tests should lock in that behavior.^[9]

8.5 Native runner and release gates

The supplied runner loads the canonical modular package before reading the test file, records source hashes and kernel information, treats aborted/empty reports as failures, and can optionally discover all repository `.wlt` files. It is a prospective tool: no output file claiming native execution is included in this bundle.

A production release gate should run the full suite in isolated kernels, alongside the standalone builder tests and loading checks. Include the declared minimum Wolfram version and at least the actual supported release/platform combinations. Avoid a test strategy that depends on session state accumulated by earlier suites. Test IDs should be unique and meaningful, and a changed public head should trigger saved-object compatibility tests.

Keep the repository's existing before/after source hashing and failure details. Add a release-level manifest that connects the tested canonical sources, generated standalone artifact, documentation examples, version metadata, and downloadable release checksum. A passing test report for one source hash must not be silently reused after regenerating or editing another artifact.^[16, 18]

8.6 How to interpret the proposed regressions

The native test file is not an assertion that the current revision passes. A01's nested path and A04's ambient-assumption cases target corrected behavior. A02 expects the new conservative export cap. A03's strict-export test deliberately changes a currently disclosed permissive behavior. Controls preserve ordinary exports, exact zero, a positive square root, classical inversion, irrational-power inversion, complete logarithmic blocks, and certificate scope.

The local patch generator's six tests do not establish Wolfram syntax or semantic validity of the modified kernel. They test that the intended edits are placed once in a controlled fixture, that malformed anchors are rejected, and that hashing and budget handling behave as designed. Applying a patch to the actual pinned source and running a native suite remain necessary acceptance steps.

9 Proposed architecture and development roadmap

9.1 A small trusted contract layer

The most effective refactoring is not a rewrite of all expansion algorithms. Introduce a small common layer that validates a result before any adapter returns it and that every operation uses for branch and precision decisions. A normalized internal germ record could contain:

```
<|"SchemaVersion" -> 1,
  "Variable" -> x,
  "Approach" -> <|"Point" -> x0, "Direction" -> direction|>,
  "Coordinate" -> positiveSmallCoordinate,
  "AssumptionContext" -> effectiveAssumptions,
  "Domain" -> retainedDomain,
  "Representation" -> representation,
  "ErrorContract" -> errorContract,
  "Evidence" -> evidence,
  "RefinementRecipe" -> recipe,
  "Capabilities" -> capabilities|>
```

This is a design sketch, not implemented package syntax. The existing family associations can initially be adapted to it without changing user-visible expressions.

The common validator should check that a purported positive coordinate is attached to the correct approach, that retained weights and remainders are ordered under the stored context, that exactness is supported, and that branch-sensitive operations have the required evidence. It should distinguish a formal coefficient identity, an asymptotic theorem application, a declared hypothesis, numerical evidence, and an interval certificate. A single Boolean **"Verified"** would erase those distinctions.

9.2 An explicit error-contract algebra

Represent error information with distinct kinds: exact zero; power-log envelope; a finite sum of carrier-weighted envelopes; finite sector remainder; and a derivative contract attached to a particular envelope and coordinate. Optional explicit constants and validity regions should be separate from an asymptotic class with an unknown constant.

Central operations should include addition, multiplication, pullback through a proved coordinate map, differentiation with an admitted derivative order, and a branch-aware observable transport rule. A comparison function should return proved dominance, proved nondominance, or unknown. Unknown must not collapse to true because a simplifier failed or a numerical estimate looked plausible.

This layer can eliminate duplicated logic between ordinary jets and composite envelopes. It also provides a natural place to fix A01: the power operation must request a branch certificate or a known positive leading unit. An adapter should not be able to bypass that requirement by calling a lower-level coefficient routine directly.

9.3 Captured assumptions and proof context

Capture the effective context at the outermost public entry. Pass it explicitly through normalization, zero tests, exact comparison, branch selection, native adapters, and refinement. Neutralize ambient **\$Assumptions** around internal evaluation. Record any additional proved or required domain restrictions when transformations are applied.

A proof-cache key should include the expression, assumptions, coordinate, direction, and relevant algorithm/version settings. An assumption formula must remain a hypothesis, not be simplified away under itself. When combining two objects, form a compatible joint context and reject inconsistent branches; do not merely retain whichever association happens to be first in the argument list.

Because Wolfram expressions can evaluate through user definitions, a held public boundary is only one part of this policy. Clearly define which inputs are evaluated normally, which callable syntax is preserved, and when own values or definitions are captured. The current held inverse syntax and normalizer are valuable starting points, but assumptions deserve the same explicit treatment.[7, 10]

9.4 A clean native adapter boundary

Expose conversion as an operation with a plan and a loss report. A possible result is a native value plus a sidecar describing dropped branch/domain/error information, or **Missing** when strict conversion is impossible. Conversion must never allocate a dense representation before checking its size.

For native special-function imports, retain the existing conservative handling of structured errors, real-domain proofs, and exact carriers. Add a trace identifying which native call supplied the expansion, its working order, the accepted remainder comparison, and the final representation. Do not infer exactness merely because a native result happened to contain no **SeriesData**; the inspected equality check is an important safeguard to preserve.[13]

9.5 Capabilities and request normalization

A proposed **SeriesCapabilities[s]** should describe supported arithmetic, composition, differentiation depth, refinement sources, numerical checking, certificate support, and conversion restrictions. The result should be machine-readable as well as useful in a notebook summary. This avoids brittle external tests against family-name strings.

Normalize requests into explicit order semantics before dispatch. A result should say, for example, “exclusive power weight h in $1/X$, with complete polynomials in $1/\log X$,” or “inclusive sector degree n with exclusive inner cutoff h .” Existing compact syntax can remain, but it should map to that single request model. Record requested and achieved precision separately, including the first known omitted block and any input precision cap.

9.6 Development stages and acceptance criteria

Stage	Work	Acceptance criterion
1: Correctness	Fix A01 and A04; cap dense export; define strict/permissive conversion.	Native path-equivalence and assumption-provenance regressions pass; optional export cannot exhaust the sparse request budget.
2: Release trust	Full native suite, source/artifact manifests, schema compatibility, license metadata alignment.	Each release links tested source hashes, generated artifact hashes, kernel/platform matrix, and explicit unsupported cases.
3: Contract consolidation	Shared assumptions, error algebra, request normalization, capabilities.	Every constructor and operation passes common invariant tests; no family silently drops conditions.

Stage	Work	Acceptance criterion
4: Measured perfor- mance	Profile comparisons, frontiers, Gamma/Barnes recurrences, adapter dispatch, and object storage.	Public-call speedups preserve exact coefficients, branch conditions, and error strength; memory is reported.
5: Focused extensions	Fill operation-closure gaps and add high-value admitted families.	Each extension has a mathematical admission theorem, precise remainder semantics, and adversarial tests.

These stages are ordered by dependency, not by a promised delivery schedule. A small correctness-focused release is more valuable than simultaneously changing the object schema, dispatch architecture, and expansion families without a full native test gate.

9.7 High-value mathematical extensions

Closure within existing scales. Before adding more named functions, make compatible logarithmic, Gamma/Barnes, exact-core, and composite objects easier to compose and refine. Preserve exact carrier information whenever expanding it prematurely would lose the requested error hierarchy.

Coupled implicit equations and varying parameters. The current inverse-expression layer accepts selected affine output-family reductions; genuinely coupled varying parameters are rejected. A natural extension is a parameter-dependent implicit solver with a proved nonsingular Jacobian in an explicit coefficient ring. Singular or coalescing regimes need a different balance and should not be treated as an ordinary extension of a fixed-parameter formula.^[10]

Uniform regimes and resonances. When exponents depend on parameters, gaps can vanish or change order, and a fixed truncation region can become nonuniform. Add parameter strata with proved order relations and identify transition regimes explicitly. A request with $\delta \rightarrow 0$ together with $u \rightarrow 0$ may require keeping $\delta \log u$ as a new scale variable.

More certificate languages. Extend exact enclosures to useful special functions only when an independent tail or interval theorem is available. Export a compact proof trace so another checker can replay the enclosure arithmetic. Keep asymptotic family certificates distinct from certificates of the stored explicit function; an unknown Big-O family needs quantitative constants and a validity interval before interval enclosure is possible.

Complex sectors and Stokes behavior. A genuine complex transseries extension would need sector domains, branch determinations, exponential dominance orders, and Stokes data or connection information where relevant. This is not achieved by replacing “real” with “complex” in existing positivity checks. It is a worthwhile research direction, but it should be a distinct capability with its own contracts rather than an overbroad claim attached to the current real engine.

Discrete and combinatorial interfaces. Native discrete asymptotics already provide a broad baseline. A package extension would be most valuable when it transports a discrete asymptotic expansion into a selected continuous inverse or retains exponentially small arithmetic contributions with explicit conventions. Avoid an ambiguous “inverse sequence” API until interpolation or generalized-inverse semantics are specified.^[27]

9.8 A recommended product description

A defensible description is: *a Wolfram Language toolkit for selected real generalized asymptotic expansions and inverse germs, with exact-weight coefficient calculus, explicit error contracts, and reusable branch-aware operations.* It should not claim to solve every transseries inversion problem or to supersede the host language’s full asymptotic ecosystem.

That narrower description is not a limitation of ambition. It identifies the part worth making exceptionally reliable. A rigorous common contract layer, transparent native cooperation, and a reproducible validation matrix would make the existing mathematical work considerably easier to trust and extend.

10 Accompanying artifacts and reproduction

10.1 Bundle contents

The archive contains the article’s complete LaTeX source tree and compiled PDF; an independent mathematical checker and its JSON result; a hash-gated patch generator; six synthetic-fixture unit tests with their execution log; fifteen prospective Wolfram tests; a native runner; a benchmark harness; a findings ledger; a source-coverage manifest; references; and checksums. No repository checkout or licensed Wolfram runtime is bundled.

article/	article.tex, article.pdf, sections/, sources.tex
code/	check_mathematics.py, propose_core_patch.py run_audit.wls, benchmark_audit.wls
tests/	audit_regressions.wlt, test_patch_planner.py
evidence/	mathematical_checks.json, patch_planner_tests.txt review_manifest.json, findings.csv source_coverage.csv, references.json, pdf_qa.json
README.md	requirements.txt, SHA256SUMS

10.2 Re-running the executed checks

From the bundle root, using Python with SymPy installed:

```
python code/check_mathematics.py \
  --output evidence/mathematical_checks.json
python -m unittest discover -s tests -p test_patch_planner.py -v
```

The recorded mathematical run used Python 3.13.5 and SymPy 1.14.0. The checker does not allocate the huge native lattice used in A02’s motivating example; it computes the number of slots symbolically with exact integer/rational arithmetic.

10.3 Generating the proposed patch

Use a local checkout of the pinned revision. The patch tool accepts the repository root or its canonical kernel file:

```
python code/propose_core_patch.py /path/to/Asymptotic \
  > proposed_core.diff
```

It compares the LF-normalized canonical file with Git blob hash

ea9eaf4a11130e922ac4fb3faae37fd8e8d29643.

A mismatch is an intentional refusal, not permission to disable the check. Review the diff, apply it in a separate working branch, and test it natively. The alternative `-output` option writes a new file without replacing an existing one; that file is not a standalone package without its companion modules.

After editing canonical kernel sources, regenerate and verify the standalone distribution with the repository's existing builder. Do not independently edit both the modular source and generated distribution.^[18]

10.4 Running native regressions later

The native scripts require a local Wolfram Language 15.0+ installation and a local checkout. An example shell invocation is:

```
export ASYMPOTIC_AUDIT_ROOT=/path/to/Asymptotic
wolframscript -file code/run_audit.wls

# Optional: include the repository's full discovered .wlt suite.
export ASYMPOTIC_AUDIT_FULL_SUITE=1
wolframscript -file code/run_audit.wls
```

PowerShell equivalents are in the README. Run baseline and modified sources separately, retain both reports, and compare the recorded source hashes. Do not interpret expected failures of new corrected-behavior tests on the baseline as failures of the test runner itself.

For benchmarks, select one case per fresh kernel through `ASYMPTOTIC_AUDIT_CASE`. The output records inputs and full results to support a semantic comparison before timing interpretation. No native benchmark results are included because none were executed during the review.

10.5 Rebuilding the article

From the `article/` directory:

```
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
```

A third pass may be needed after changes to cross-references or the contents. The source uses standard LaTeX packages and no custom font files. The PDF was compiled and visually checked for this delivery; its existence does not imply native Wolfram execution.

10.6 Final assessment

The repository deserves further development. Its exact coefficient calculus, explicit real branches, conservative treatment of several remainder hazards, and specialized inverse cores are more substantial than a wrapper around one native command. Its strongest future direction is to become a dependable contract-preserving layer over both its own algorithms and Wolfram's native asymptotics.

The immediate priorities are concrete: centralize the fractional-power branch invariant, capture assumptions faithfully, prevent optional export from defeating sparse resource limits, and make native conversion loss explicit. Then validate the complete release surface and consolidate the increasingly heterogeneous representations and request semantics. The supplied examples, tests, and patch planner are intended to make those recommendations actionable without overstating what was executed or proved.

A Source coverage and limitations

The following is a review-coverage ledger, not a claim that every listed file was fully verified. Repository paths are relative to the pinned revision. “Inspected” means the source text was read; it does not mean that every branch was executed or formally proved correct.

Source	Read coverage	Main use
Kernel/AsymptoticInverse.wl	Successive ranges across the main module, plus focused re-reads	Jet algebra, inversion, error transport, conversion, properties, dispatch.
Kernel/SeriesOperations.wl	Lines 1–320, with overlaps	Dedicated and recursive observable paths; branch guard comparison.
Kernel/SeriesArithmetic.wl	Lines 1–290	Held normalization, arithmetic dispatch, composite fallback, resource policy.
Kernel/InverseFunctionExpressions.wl	Lines 1–130 and 160–end	Public wrappers, assumptions, branch provenance, recursive inverse composition.
Kernel/InverseCertificates.wl	Successive ranges through the full file	Exact interval arithmetic, domain proofs, residual brackets, scope and options.
Kernel/IncrementalInverse.wl	Lines 1–155	Frontier/state design, cache policy, Newton invariants, grouped-route rationale.
Kernel/SourceCoordinates.wl	Lines 1–140	Source chart admission and reconstruction constraints.
Kernel/GammaInverse.wl	Lines 1–150	Gamma/Barnes model recognition, core formulas, coefficient recurrence, bounds.
Kernel/NativeSpecialFunctions.wl	Lines 1–150 and 250–end	Native tree error transport, branch and real-domain checks, result metadata.
Documentation/UserGuide.md	Lines 1–300	Current public overview, loading, order/branch conventions, result properties and native-export warning.
PacletInfo.wl	Full file	Version, minimum kernel, extensions, license metadata.
README.md	Connector read	Repository orientation and declared scope.

Source	Read coverage	Main use
<code>validation/README.md</code>	Lines 1–150	Focused validation record and attributed timings.
<code>validation/FocusedTests.wl</code>	Full file	Test-runner design, failure handling, source hashes.
<code>.github/workflows/standalone.yml</code>	Full file	Checked-in standalone freshness/build-test workflow.
<code>docs/development/README.md</code>	Full returned text	Canonical/generated source relationship, loading history, validation workflow.
<code>article/asymptotic-inverse.tex</code>	Top-level source	Mathematical article scope and organization; included proofs not all audited.
<code>LICENSE</code>	Full file	MIT No Attribution text and metadata consistency.
Repository tree and branch metadata	Inventory and revision responses	Immutable revision, file layout, canonical/distribution distinction.

Paths beginning `Kernel/`, `Documentation/`, or `PacletInfo.wl` in this table are under `AsymptoticInverse/`. Other paths are repository-relative as written. Source range endpoints are review windows, not necessarily function boundaries.

Files reached only through module-load lists or public guide summaries were not silently counted as fully reviewed. In particular, the entire flat-sector, Fourier, logarithmic-scale, branch-selector, refinement, and Barnes-specific implementation families were not independently traced line by line. The article makes architectural recommendations about these families from their documented contracts and inspected integration points, but it does not claim an exhaustive defect inventory for them.

Official Wolfram documentation was consulted for the host-language comparison and assumption semantics, and NIST DLMF for asymptotic, Gamma, and Barnes background. These are current primary references, not evidence that a particular native call was successfully executed. The bibliography’s URLs make the distinction between pinned repository evidence and mutable official documentation explicit.

References

- [1] Asymptotic Contributors / V. Reshetnikov. *Pinned revision and commit metadata*. Revision 07a9781, inspected September 9, 2026. <https://github.com/VladimirReshetnikov/Asymptotic/commit/07a9781212beb2eeb9ff16aa625b50ac27974078>
- [2] Asymptotic Contributors / V. Reshetnikov. *Pinned repository tree*. Revision 07a9781, inspected September 9, 2026. <https://github.com/VladimirReshetnikov/Asymptotic/tree/07a9781212beb2eeb9ff16aa625b50ac27974078>
- [3] Asymptotic Contributors / V. Reshetnikov. *Repository README*. Revision 07a9781, inspected September 9, 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/README.md>
- [4] Asymptotic Contributors / V. Reshetnikov. *Paclet metadata*. Revision 07a9781, inspected September 9, 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/PacletInfo.wl>
- [5] Asymptotic Contributors / V. Reshetnikov. *Canonical main kernel*. Revision 07a9781, inspected September 9, 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/AsymptoticInverse.wl>
- [6] Asymptotic Contributors / V. Reshetnikov. *Series operations*. Revision 07a9781, inspected September 9, 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/SeriesOperations.wl>
- [7] Asymptotic Contributors / V. Reshetnikov. *Arithmetic and held normalization*. Revision 07a9781, inspected September 9, 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/SeriesArithmetic.wl>
- [8] Asymptotic Contributors / V. Reshetnikov. *Incremental inverse states and Newton refinement*. Revision 07a9781, inspected September 9, 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/IncrementalInverse.wl>
- [9] Asymptotic Contributors / V. Reshetnikov. *Exact inverse certificates*. Revision 07a9781, inspected September 9, 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/InverseCertificates.wl>
- [10] Asymptotic Contributors / V. Reshetnikov. *Applied inverse expressions and public wrappers*. Revision 07a9781, inspected September 9, 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/InverseFunctionExpressions.wl>
- [11] Asymptotic Contributors / V. Reshetnikov. *Source coordinate transformations*. Revision 07a9781, inspected September 9, 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/SourceCoordinates.wl>
- [12] Asymptotic Contributors / V. Reshetnikov. *Gamma and Barnes inverse construction*. Revision 07a9781, inspected September 9, 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/GammaInverse.wl>
- [13] Asymptotic Contributors / V. Reshetnikov. *Native special-function expansion adapter*. Revision 07a9781, inspected September 9, 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/NativeSpecialFunctions.wl>
- [14] Asymptotic Contributors / V. Reshetnikov. *Current user guide*. Revision 07a9781, inspected September 9, 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Documentation/UserGuide.md>
- [15] Asymptotic Contributors / V. Reshetnikov. *Validation record*. Revision 07a9781, inspected September 9, 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/validation/README.md>
- [16] Asymptotic Contributors / V. Reshetnikov. *Focused native test runner*. Revision 07a9781, inspected September 9, 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/validation/FocusedTests.wl>

- [17] Asymptotic Contributors / V. Reshetnikov. *Standalone freshness workflow*. Revision 07a9781, inspected September 9, 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/.github/workflows/standalone.yml>
- [18] Asymptotic Contributors / V. Reshetnikov. *Development and standalone provenance notes*. Revision 07a9781, inspected September 9, 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/docs/development/README.md>
- [19] Asymptotic Contributors / V. Reshetnikov. *Mathematical article, top-level LaTeX source*. Revision 07a9781, inspected September 9, 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/article/asymptotic-inverse.tex>
- [20] Asymptotic Contributors / V. Reshetnikov. *Repository license text*. Revision 07a9781, inspected September 9, 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/LICENSE>
- [21] Wolfram Research. *Series*. Official reference, accessed September 9, 2026. <https://reference.wolfram.com/language/ref/Series.html>
- [22] Wolfram Research. *SeriesData*. Official reference, accessed September 9, 2026. <https://reference.wolfram.com/language/ref/SeriesData.html>
- [23] Wolfram Research. *InverseSeries*. Official reference, accessed September 9, 2026. <https://reference.wolfram.com/language/ref/InverseSeries.html>
- [24] Wolfram Research. *ComposeSeries*. Official reference, accessed September 9, 2026. <https://reference.wolfram.com/language/ref/ComposeSeries.html>
- [25] Wolfram Research. *Asymptotic*. Official reference, accessed September 9, 2026. <https://reference.wolfram.com/language/ref/Asymptotic.html>
- [26] Wolfram Research. *AsymptoticSolve*. Official reference, accessed September 9, 2026. <https://reference.wolfram.com/language/ref/AsymptoticSolve.html>
- [27] Wolfram Research. *DiscreteAsymptotic*. Official reference, accessed September 9, 2026. <https://reference.wolfram.com/language/ref/DiscreteAsymptotic.html>
- [28] Wolfram Research. *AsymptoticLess*. Official reference, accessed September 9, 2026. <https://reference.wolfram.com/language/ref/AsymptoticLess.html>
- [29] Wolfram Research. *O*. Official reference, accessed September 9, 2026. <https://reference.wolfram.com/language/ref/O.html>
- [30] Wolfram Research. *Assuming*. Official reference, accessed September 9, 2026. <https://reference.wolfram.com/language/ref/Assuming.html>
- [31] Wolfram Research. *Simplify*. Official reference, accessed September 9, 2026. <https://reference.wolfram.com/language/ref/Simplify.html>
- [32] Wolfram Research. *Assumptions*. Official reference, accessed September 9, 2026. <https://reference.wolfram.com/language/ref/Assumptions.html>
- [33] NIST Digital Library of Mathematical Functions. *Section 2.1: Definitions and elementary properties of asymptotic approximations*. Official reference, accessed September 9, 2026. <https://dlmf.nist.gov/2.1>
- [34] NIST Digital Library of Mathematical Functions. *Section 5.11: Gamma asymptotic expansions*. Official reference, accessed September 9, 2026. <https://dlmf.nist.gov/5.11>
- [35] NIST Digital Library of Mathematical Functions. *Section 5.17: Barnes G-function*. Official reference, accessed September 9, 2026. <https://dlmf.nist.gov/5.17>
- [36] SPDX License List. *MIT No Attribution (MIT-0)*. Official reference, accessed September 9, 2026. <https://spdx.org/licenses/MIT-0.html>